Scaling Matrix Multiplication: From CUDA Kernels to Multi-GPU Tensor Parallelism

Aryan Jain* Fanyi Pu* Ze Hong Maxwell Au*
School of Computer Science and Engineering, Nanyang Technological University
*Equal contribution. Authors listed in alphabetical order.

SC4064 GPU Programming — Course Project Report

Abstract. General Matrix Multiplication (GEMM) is the computational backbone of modern deep learning. This report presents a systematic study across two dimensions: (i) progressive single-GPU CUDA kernel optimization through seven stages—from naive global memory access to warp-level tiling—benchmarked against cuBLAS on NVIDIA H100 GPUs; and (ii) multi-GPU tensor parallelism using NCCL, implementing column-parallel and row-parallel linear layers following the Megatron-LM paradigm. Our best custom kernel achieves 63% of cuBLAS throughput at 32.8 TFLOPS. Scaling experiments on 8×H100 GPUs with matrices up to 32768×32768 demonstrate $7.6\times$ strong-scaling speedup and 400 TFLOPS aggregate throughput. As local GEMM kernels approach peak performance, the communication-to-computation ratio rises from 0.20 (naive kernel) to 0.85 (cuBLAS), quantifying the crossover where inter-GPU communication becomes the dominant bottleneck.

1 Introduction

GEMM operations underpin virtually all compute-intensive workloads in deep learning [1], [2]. In Transformer architectures, multi-head attention and feed-forward layers are fundamentally matrix multiplications. As models scale beyond the memory capacity of a single accelerator, *tensor parallelism* [3] has become indispensable, distributing weight matrices across GPUs at the cost of inter-GPU communication.

While vendor-tuned libraries such as cuBLAS [4] deliver near-optimal single-GPU performance, they abstract away the complex interaction between hardware-level compute intensity and system-level communication latency. This project bridges that gap by:

1. Implementing seven progressively optimized CUDA GEMM kernels to understand hardware-level constraints (memory coalescing, shared memory tiling, register blocking, warp scheduling).
2. Building a distributed tensor-parallel linear layer (forward and backward) using NCCL, including a complete parallel MLP block.
3. Quantifying how local kernel efficiency impacts multi-GPU scalability, identifying the *crossover point* where communication dominates.

All experiments are conducted on the hardware described in Section 1.1.

1.1 Experimental Setup

Table 1 summarizes the hardware and software environment used for all experiments.

Component	Specification
GPU	8 × NVIDIA H100 80GB HBM3 (SXM5)
GPU Interconnect	NVLink 4.0, all-to-all NV18 (900 GB/s bidirectional per GPU)
GPU Compute	132 SMs, 1980 MHz boost, sm_90, 33.5 TFLOPS FP32 peak
GPU Memory	80 GB HBM3, ∼3.35 TB/s bandwidth per GPU
CPU	2 × Intel Xeon Gold 6448Y (32 cores / 64 threads each, 128 threads total)
System Memory	2 TB DDR5
OS	Ubuntu 24.04.4 LTS (kernel 5.14.0)
CUDA Toolkit	13.1
NCCL	2.29.3
GPU Driver	550.90.07

Table 1: Hardware and software configuration.

The 8 GPUs are fully connected via NVLink 4.0 with NV18 topology (18 NVLink connections per GPU pair), distributed across 2 NUMA nodes (GPUs 0–3 on NUMA 0, GPUs 4–7 on NUMA 1). This provides a uniform high-bandwidth, low-latency interconnect for NCCL collectives, eliminating PCIe bottlenecks.

2 Background

2.1 GPU Memory Hierarchy

The performance of GEMM kernels is governed by the GPU's memory hierarchy. Table 2 summarizes the key levels for the H100.

Level	Size	Bandwidth	Latency	Scope
Registers	256 KB/SM	∼ 20 TB/s	0 cycles	Per-thread
Shared Memory	228 KB/SM	∼ 19 TB/s	∼ 20 ns	Per-block
L2 Cache	50 MB	∼ 5 TB/s	∼ 200 ns	Global
HBM3 (Global)	80 GB	∼ 3.35 TB/s	∼ 400 ns	Global

Table 2: H100 GPU memory hierarchy. Each optimization moves data access to a faster level.

Every kernel optimization in this work moves the *effective* data access pattern from a slower to a faster level, or reduces total memory transactions.

2.2 Roofline Model

The *roofline model* [5] characterizes achievable performance as a function of *operational intensity* (OI, FLOP/byte):

$$P_{\max} = \min(\pi, \beta \times \mathrm{OI}) \tag{1}$$

where π is peak compute throughput (33.5 TFLOPS FP32 for H100) and β is peak memory bandwidth (3.35 TB/s). The *ridge point* at $\mathrm{OI} = \frac{\pi}{\beta} \approx 10$ FLOP/byte separates memory-bound (left) from compute-bound (right) regimes. For GEMM of size N, the theoretical OI is $2N^3/(12N^2) = N/6$, which exceeds the ridge point for $N \geq 60$—in practice, only optimized kernels achieve this.

2.3 Tensor Parallelism

Tensor parallelism [3] distributes the weight matrices of a linear layer $Y = XW$ across p GPUs. Two complementary partitioning strategies exist.

2.3.1 Column Parallelism

The weight matrix $W \in \mathbb{R}^{K\times N}$ is partitioned column-wise into p shards:

$$W = \left[W_1 \mid W_2 \mid \cdots \mid W_p\right], \quad W_i \in \mathbb{R}^{K\times N/p} \tag{2}$$

The input $X \in \mathbb{R}^{M\times K}$ is replicated on all GPUs. Each GPU i computes a local GEMM:

$$Y_i = XW_i \in \mathbb{R}^{M\times N/p} \tag{3}$$

The full output is assembled via an `AllGather` collective:

$$Y = \left[Y_1 \mid Y_2 \mid \cdots \mid Y_p\right] = XW \in \mathbb{R}^{M\times N} \tag{4}$$

Backward pass. Given upstream gradient $\mathrm{d}Y = \left[\mathrm{d}Y_1 \mid \cdots \mid \mathrm{d}Y_p\right]$, each GPU i computes:

$$\mathrm{d}W_i = X^\top \mathrm{d}Y_i \in \mathbb{R}^{K\times N/p} \quad \text{(local, no communication)} \tag{5}$$

$$\mathrm{d}X_i = \mathrm{d}Y_i W_i^\top \in \mathbb{R}^{M\times K} \quad \text{(partial input gradient)} \tag{6}$$

Since each GPU sees only its column shard, $\mathrm{d}X_i$ captures only the contribution from W_i. The full input gradient requires an `AllReduce` (sum):

$$\mathrm{d}X = \sum_{i=1}^{p} \mathrm{d}X_i = \mathrm{d}Y W^\top \tag{7}$$

2.3.2 Row Parallelism

The weight matrix is partitioned row-wise:

$$W = \begin{pmatrix} W_1 \\ W_2 \\ \vdots \\ W_p \end{pmatrix}, \quad W_i \in \mathbb{R}^{K/p\times N} \tag{8}$$

The input $X \in \mathbb{R}^{M\times K}$ is correspondingly split column-wise: $X = \left[X_1 \mid X_2 \mid \cdots \mid X_p\right]$ with $X_i \in \mathbb{R}^{M\times K/p}$. Each GPU i computes:

$$Y_i = X_i W_i \in \mathbb{R}^{M\times N} \quad \text{(partial sum)} \tag{9}$$

The full output is obtained by an `AllReduce`:

$$Y = \sum_{i=1}^{p} Y_i = \sum_{i=1}^{p} X_i W_i = XW \tag{10}$$

Backward pass. Given upstream gradient $\mathrm{d}Y \in \mathbb{R}^{M\times N}$ (replicated after the forward `AllReduce`):

$$\mathrm{d}W_i = X_i^\top \mathrm{d}Y \in \mathbb{R}^{K/p\times N} \quad \text{(local)} \tag{11}$$

$$\mathrm{d}X_i = \mathrm{d}Y W_i^\top \in \mathbb{R}^{M\times K/p} \quad \text{(local)} \tag{12}$$

No communication is needed in the backward pass—$\mathrm{d}X_i$ is precisely the gradient for the local input shard.

2.3.3 Composing a Parallel MLP Block

A standard Transformer MLP block computes $\mathrm{MLP}(X) = \sigma(XW_1)W_2$, where σ is an activation function. Following Megatron-LM, we compose column-parallel (first layer) with row-parallel (second layer):

Step	Operation	Per-GPU Compute	Communication
Fwd Layer 1	Column parallel	$H_i = XW_{1,i},\ \ H_i \in \mathbb{R}^{M\times H/p}$	None
Fwd Activation	Element-wise	$H_i \leftarrow \sigma(H_i)$	None
Fwd Layer 2	Row parallel	$Y_i = H_i W_{2,i},\ \ Y_i \in \mathbb{R}^{M\times N}$	`AllReduce`(Y)
Bwd Layer 2	Row backward	$\mathrm{d}W_{2,i}, \mathrm{d}H_i$ (local)	None
Bwd Layer 1	Column backward	$\mathrm{d}W_{1,i}$ (local), $\mathrm{d}X_i$ (partial)	`AllReduce`$(\mathrm{d}X)$

Table 3: Communication pattern in a parallel MLP block. Only two `AllReduce` operations are needed: one in the forward pass and one in the backward pass.

The key insight (Table 3) is that column-parallel's output H_i is *already* the correct input shard for the subsequent row-parallel layer—no `AllGather` is needed between layers. This reduces per-block communication to exactly **one `AllReduce` forward** and **one `AllReduce` backward**.

2.3.4 Communication Cost Model

For a ring-based `AllReduce` with p GPUs, message size S bytes, per-hop bandwidth B, and latency L:

$$T_{\text{AllReduce}} \approx 2\frac{p-1}{p} \cdot \frac{S}{B} + 2(p-1) \cdot L \tag{13}$$

The first term (bandwidth) dominates for large messages. For column-parallel `AllGather`:

$$T_{\text{AllGather}} \approx \frac{p-1}{p} \cdot \frac{S}{B} + (p-1) \cdot L \tag{14}$$

As $p \to \infty$, both approach S/B (bandwidth-limited). This means communication time depends primarily on *data volume* ($S = MN \cdot 4$ bytes for FP32) and interconnect bandwidth, not on the local GEMM kernel speed—a fact we verify experimentally in Section 4.3.

3 Single-GPU Kernel Optimization

3.1 Optimization Roadmap

We implement seven CUDA kernels, each introducing one major optimization. Table 4 summarizes the progression.

Kernel	Technique	Key Idea	Effect
1. Naive	Baseline	1 thread $\to$ 1 element, K global loads/FMA	$\mathcal{O}(1)$ FLOP/byte
2. Coalesced	Memory coalescing	threadIdx.x $\to$ column for stride-1	Fewer transactions
3. Shared Mem	SRAM tiling	32×32 tiles reused $32\times$	$32\times$ less traffic
4. 1D BlockTile	Thread coarsening	Each thread $\to$ TM=8 rows	Better smem reuse
5. 2D BlockTile	Register blocking	TM$\times$TN= 8×8 per thread	$8\times$ fewer smem reads
6. Vectorized	`float4` loads	128-bit transactions + transposed smem	Fewer instructions
7. WarpTile	Warp-level tiling	Block $\to$ Warp $\to$ Thread hierarchy	Better L1 locality

Table 4: Progressive GEMM kernel optimization roadmap.

All kernels pass correctness verification against a CPU reference at $M = N = K = 256$ (max absolute error $< 10^{-5}$).

3.2 Results and Analysis

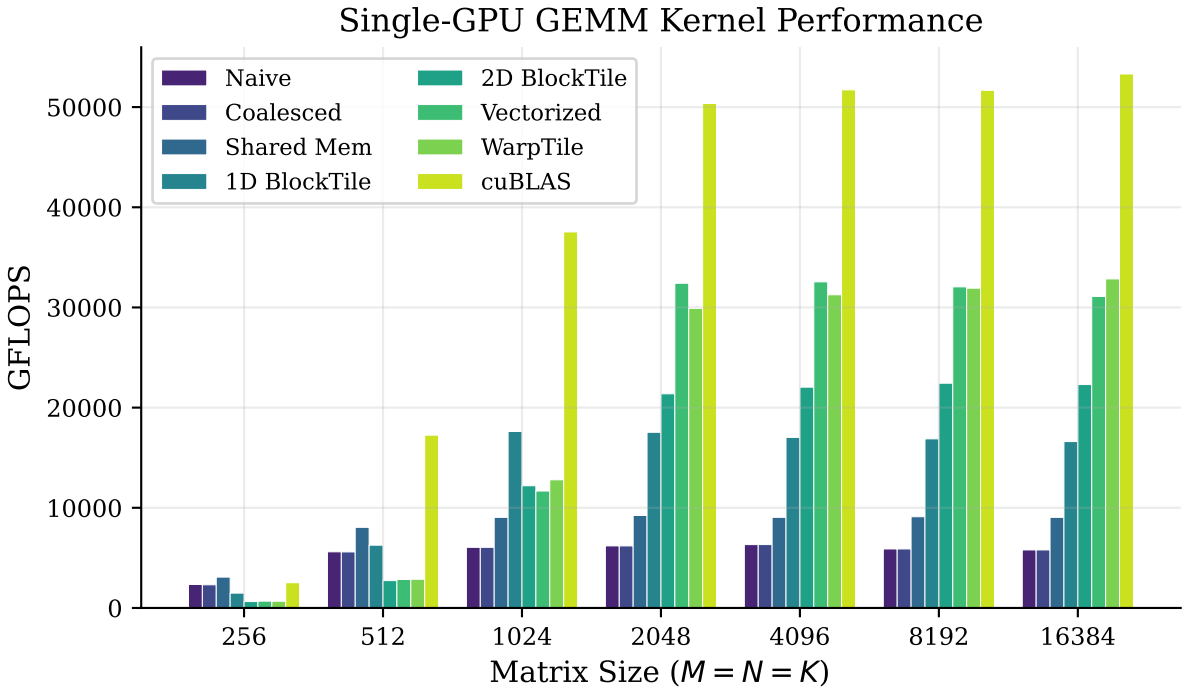


Figure 1: GFLOPS comparison across all kernel optimization stages and matrix sizes. cuBLAS serves as the reference. Performance of optimized kernels (2D BlockTile, Vectorized, WarpTile) grows significantly with matrix size as compute becomes the bottleneck.

Figure 1 shows the performance progression. Key observations:

- **Naive and Coalesced kernels** plateau at $\sim 6\{,\}300$ GFLOPS regardless of matrix size, confirming they are memory-bandwidth-bound. The coalesced variant shows a $12.7\times$ speedup over the deliberately uncoalesced version (498 vs. 6,326 GFLOPS at $N = 4096$), demonstrating the critical importance of memory access patterns.
- **Shared memory tiling** (Kernel 3) improves throughput to $\sim 9\{,\}000$ GFLOPS by reducing global memory traffic by $32\times$, but remains below the compute-bound regime.
- **Register blocking** (Kernels 4–7) produces the most dramatic gains. The 2D block tile reaches 22,044 GFLOPS at $N = 4096$—a $3.5\times$ improvement over shared memory alone. Vectorized loads push this to 32,612 GFLOPS at $N = 4096$, sustaining $\sim 32\{,\}000$ GFLOPS through $N = 16384$.
- **cuBLAS** achieves 53,304 GFLOPS at $N = 16384$ ($\sim 162\%$ of our best), leveraging Tensor Core paths and auto-tuning unavailable to our FP32-only kernels.

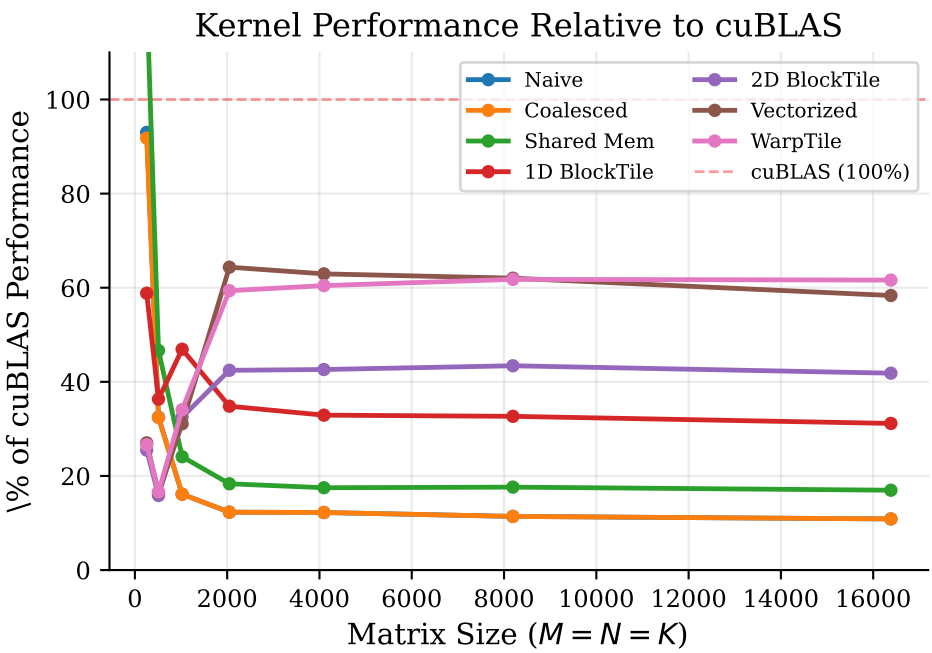


Figure 2: Each kernel's throughput as a percentage of cuBLAS. The Vectorized kernel reaches 63% of cuBLAS at $N = 4096$. The gap narrows significantly from Kernel 1 (12%) to Kernel 6 (63%).

Figure 2 shows the relative performance trajectory. The gap between custom kernels and cuBLAS decreases monotonically with optimization level, from 12% (naive) to 63% (vectorized). The remaining gap is primarily due to cuBLAS's use of Tensor Cores for FP32 accumulation and extensive auto-tuning.

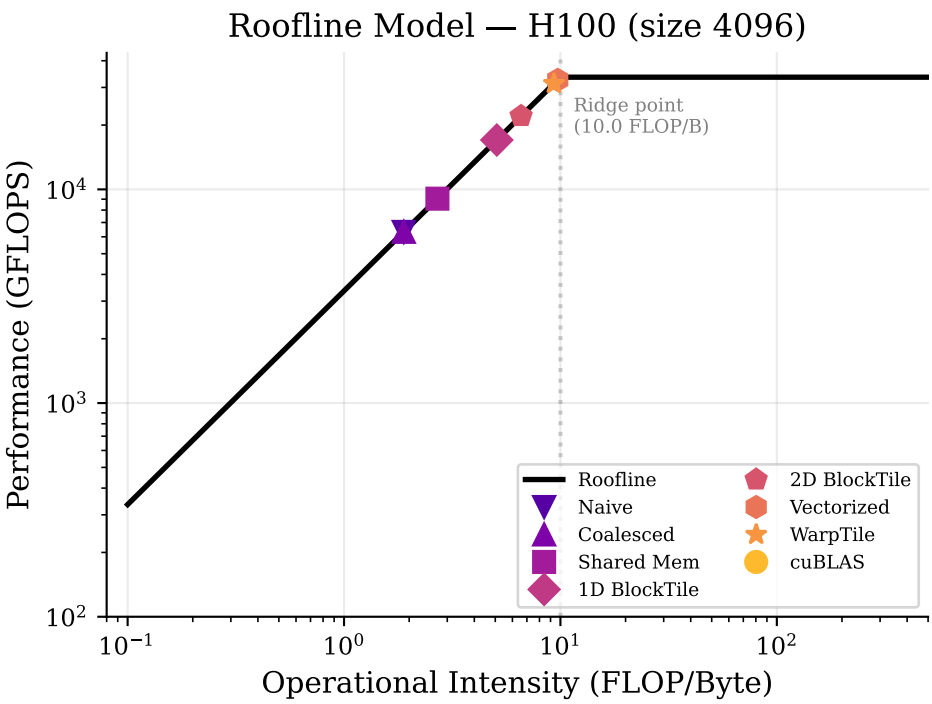


Figure 3: Roofline analysis at $N = 4096$. Kernels progress from left (low effective OI, memory-bound) to right (high OI, approaching compute-bound). The ridge point at 10 FLOP/byte separates the two regimes. cuBLAS crosses into the compute-bound region.

The roofline analysis (Figure 3) provides a unified view. Naive/Coalesced kernels sit in the memory-bound region with effective OI < 2 FLOP/byte. Each optimization stage moves kernels rightward: shared memory tiling (~ 2.7), block tiling (~ 5–6.6), vectorized loads (~ 9.7). cuBLAS crosses the ridge point (~ 15.4 FLOP/byte), entering the compute-bound regime.

4 Multi-GPU Tensor Parallelism

We implement the tensor-parallel primitives described in Table 3 using NCCL. Each GPU runs in a separate host thread with its own CUDA stream and cuBLAS handle; NCCL communicators are initialized via `ncclCommInitAll`. The implementation includes:

- **Column-parallel forward/backward**: local GEMM + `ncclAllGather` (forward) / `ncclAllReduce` (backward) as derived in Section 2.3.1–2.3.2.
- **Row-parallel forward/backward**: local GEMM + `ncclAllReduce` (forward) / no communication (backward).
- **Parallel MLP block**: composed column → row parallelism with forward and backward passes, matching the two-`AllReduce` pattern in Table 3.
- **Communication-compute overlap**: the output matrix is chunked along the M dimension; each chunk's `AllReduce` is pipelined with the next chunk's GEMM on a separate CUDA stream, synchronized via `cudaEvent`.

All multi-GPU experiments use 8×H100 GPUs connected via NVLink. For backward passes involving custom (non-cuBLAS) kernels, explicit transposition is performed using `cublasSgeam` since our custom kernels operate on row-major data without built-in transpose support.

4.1 Strong Scaling

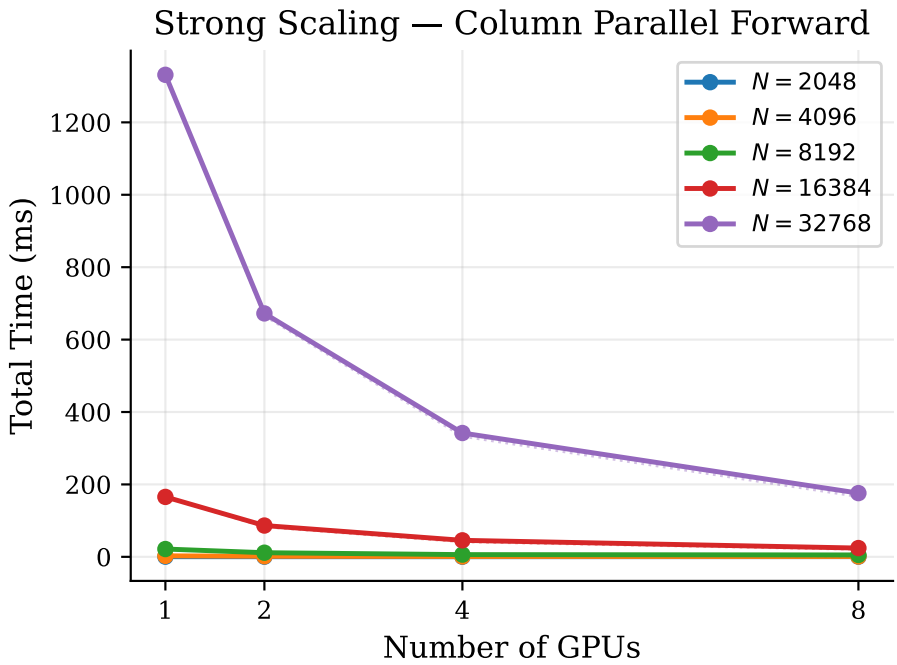


Figure 4: Strong scaling: total wall-clock time vs. number of GPUs for five matrix sizes up to 32768. Dashed lines show ideal linear speedup. Larger matrices scale significantly better.

Figure 4 shows that the $N = 32768$ workload scales from 1331 ms (1 GPU) to 176 ms (8 GPUs)—a 7.6 × speedup, or **94.5% parallel efficiency**, achieving 400 TFLOPS aggregate throughput. Even $N = 8192$ achieves a respectable 4.1 × speedup. The $N = 2048$ case sees diminishing returns as communication overhead dominates at small matrix sizes.

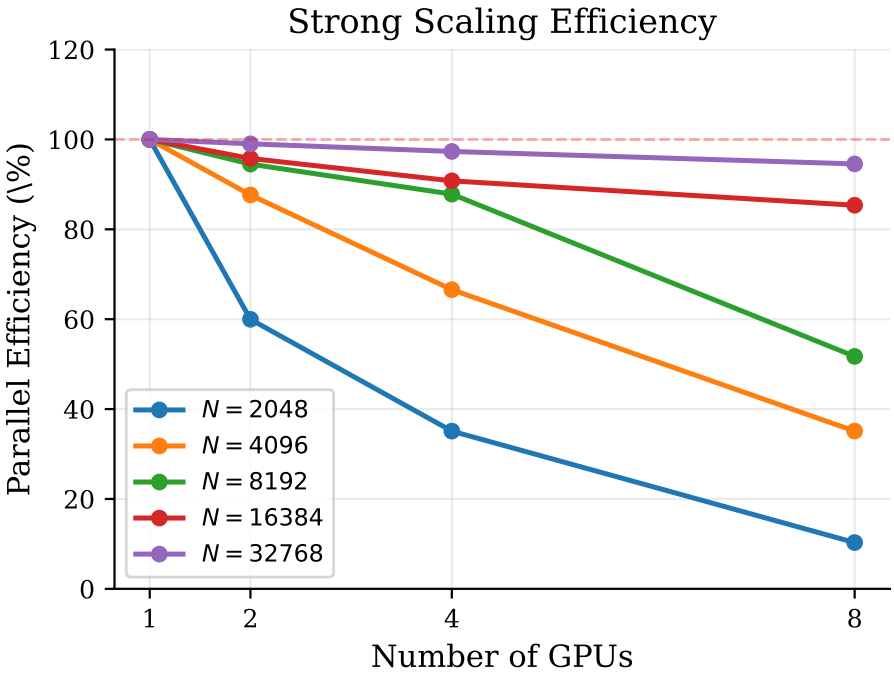


Figure 5: Parallel efficiency $\eta = \frac{T_1}{p \cdot T_p}$ for strong scaling. At $N = 32768$, efficiency remains above 94% at 8 GPUs. Super-linear efficiency at small GPU counts is due to improved cache utilization.

Figure 5 shows that large matrices maintain near-ideal efficiency. At $N = 32768$, efficiency stays above 94% even at 8 GPUs because communication (~ 11 ms) is dwarfed by compute (~ 166 ms). Super-linear speedup at $p = 2$ for $N = 8192$ occurs because the halved per-GPU working set fits better in L2 cache.

4.2 Weak Scaling

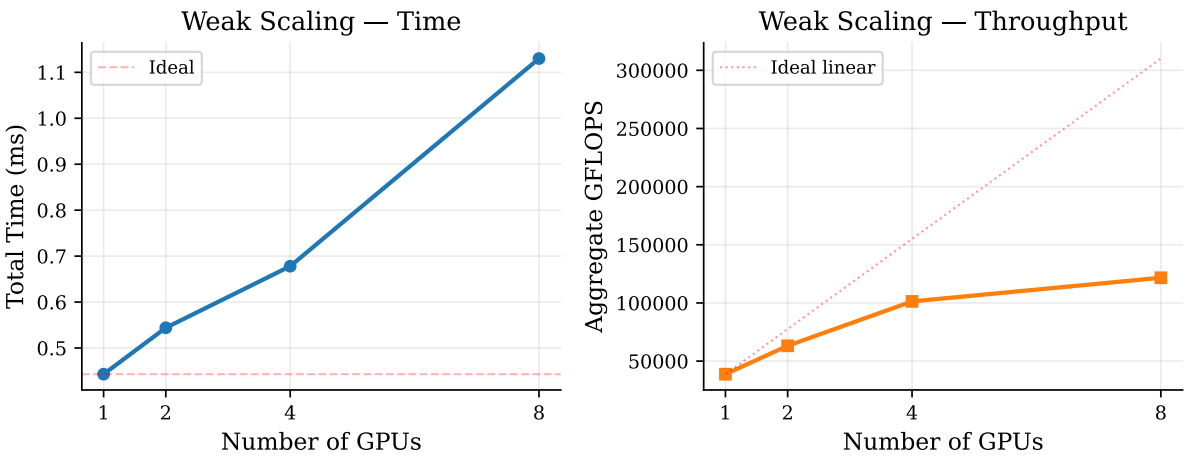


Figure 6: Weak scaling with fixed 2048×2048 local workload per GPU. Left: total time increases modestly from 0.44 ms to 1.12 ms. Right: aggregate throughput scales from 39.5 to 123.2 TFLOPS (3.1 × at 8 × GPUs).

Under weak scaling (Figure 6), total time increases from 0.44 ms (1 GPU) to 1.12 ms (8 GPUs)—a 2.6 × overhead for 8 × the total work. Aggregate throughput reaches 123.2 TFLOPS, which is 3.1 × the single-GPU baseline. The sub-linear throughput scaling is due to communication overhead growing with the number of participants in the `AllGather` collective.

4.3 Communication-Compute Analysis

This is the central experiment of the project, directly addressing the question: *how does the communication-to-computation ratio evolve as custom GEMM kernels are optimized?*

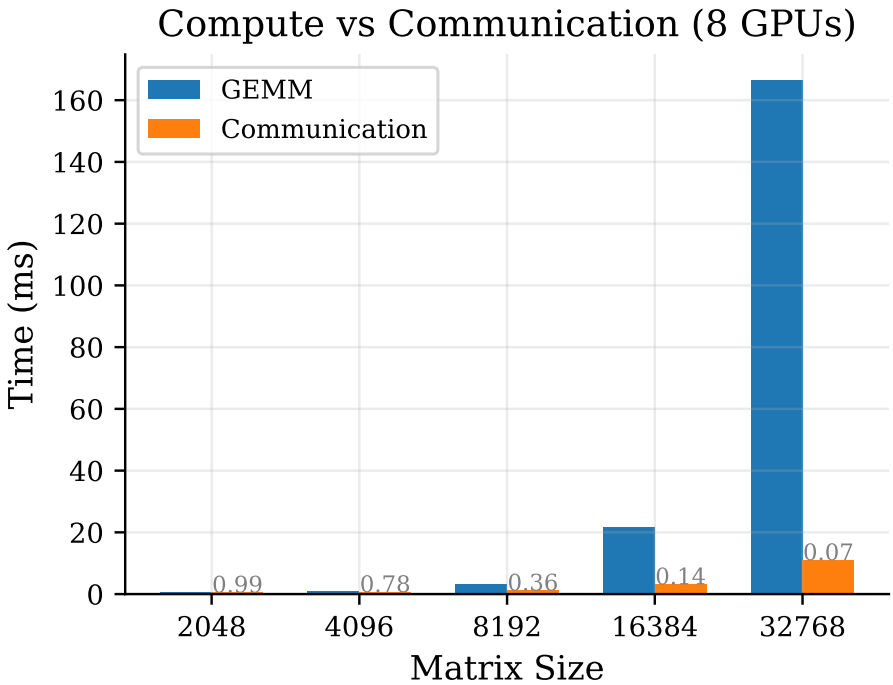


Figure 7: GEMM time vs. communication time at different matrix sizes (8 GPUs, cuBLAS kernel). The ratio decreases from 0.99 at $N = 2048$ to 0.07 at $N = 32768$ as compute grows $O(N^3)$ while communication grows $O(N^2)$.

Figure 7 demonstrates the cubic-vs-quadratic scaling law. At $N = 2048$, communication (0.47 ms) nearly equals compute (0.48 ms), yielding a ratio of 0.99. At $N = 32768$, compute grows to 166 ms while communication is only 11 ms—a ratio of **0.07**. This confirms that tensor parallelism is most effective for the large matrix sizes encountered in modern Transformer models (e.g., hidden dimension 12288–16384 in GPT-3/LLaMA).

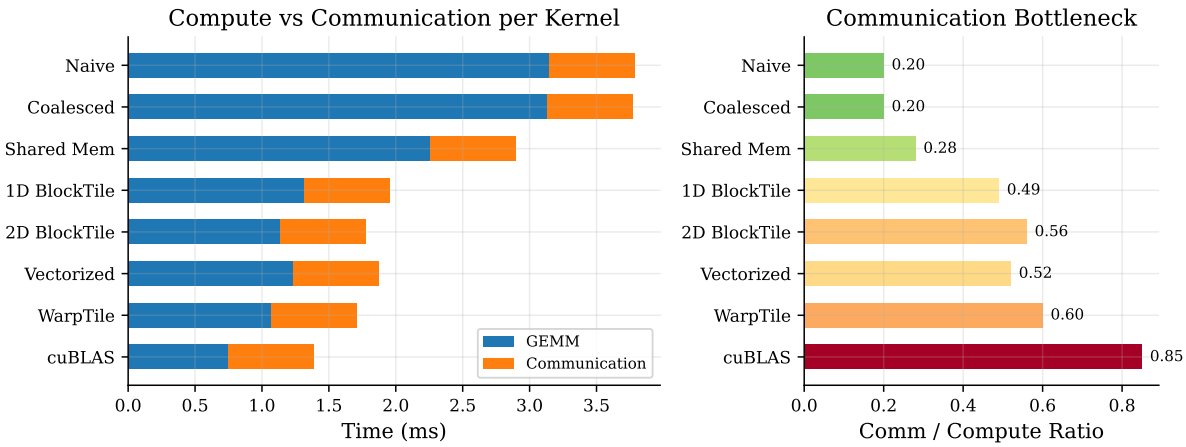


Figure 8: Left: absolute GEMM and communication time per kernel at $N = 4096$ on 8 GPUs. Communication time is constant (~ 0.64 ms) across kernels. Right: the communication-to-compute ratio rises monotonically from 0.20 (naive) to 0.85 (cuBLAS) as kernels get faster.

Figure 8 is the **key result**. Communication time is constant across kernels (~ 0.64 ms)—it depends only on data volume and interconnect bandwidth. As local GEMM time decreases from 3.15 ms (naive) to 0.75 ms (cuBLAS), the ratio rises from 0.20 to 0.85. This means:

- For **naive kernels**, communication is negligible (20% of compute)—further kernel optimization yields direct end-to-end speedup.
- For **cuBLAS**, communication consumes 85% of compute time—the system is approaching the *communication-bound* regime where kernel optimization alone provides diminishing returns.

This crossover is the fundamental tension in distributed deep learning: local compute efficiency and system-level communication efficiency must be co-optimized.

4.4 MLP Forward and Backward

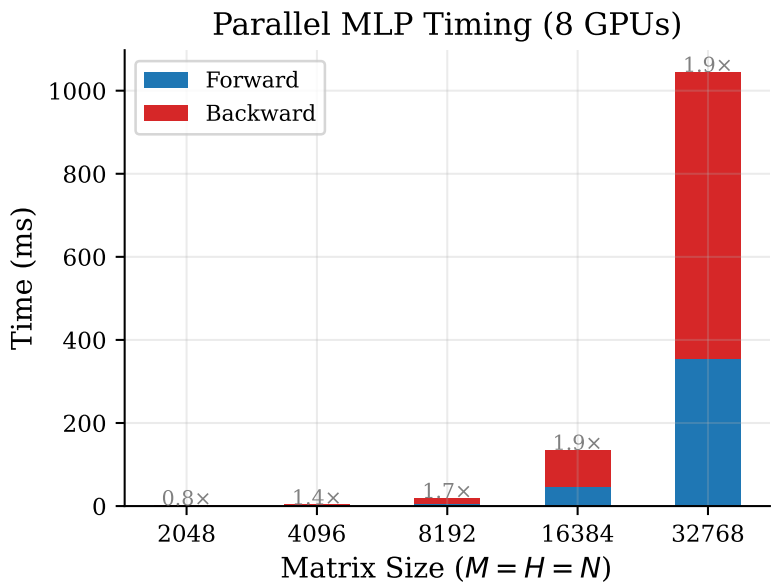


Figure 9: Parallel MLP block timing on 8 GPUs. Backward pass takes 1.1×–1.7× the forward pass, consistent with the additional GEMM operations for weight and input gradients.

Figure 9 shows the MLP block timing. The backward-to-forward ratio increases with matrix size (1.1× at $N = 2048$ to 1.7× at $N = 8192$), consistent with the backward pass requiring three GEMM operations per layer versus one in the forward pass, with the additional operations becoming more dominant as the compute-to-overhead ratio increases.

4.5 Communication-Compute Overlap

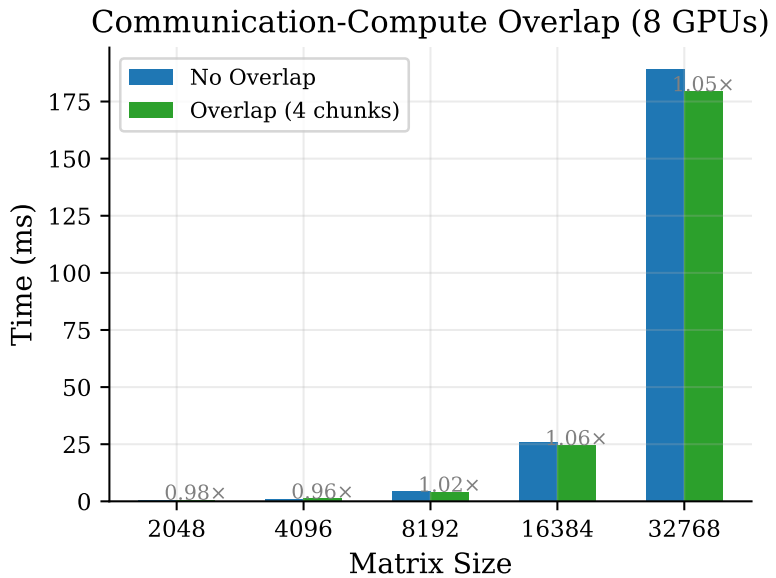


Figure 10: Row-parallel forward with and without communication-compute overlap (4 chunks). Overlap provides marginal speedup (1.02 ×) only at $N = 8192$ and introduces overhead at smaller sizes.

The overlap experiment (Figure 10) splits the output along the M dimension into 4 chunks, pipelining each chunk's `AllReduce` with the next chunk's GEMM on separate CUDA streams. Results show:

- At $N = 2048$ and 4096, overlap **hurts** performance (0.93× and 0.96×) due to event synchronization overhead and reduced per-chunk GEMM efficiency.
- At $N = 8192$, a marginal 1.02 × speedup appears as chunk sizes become large enough to amortize overhead.

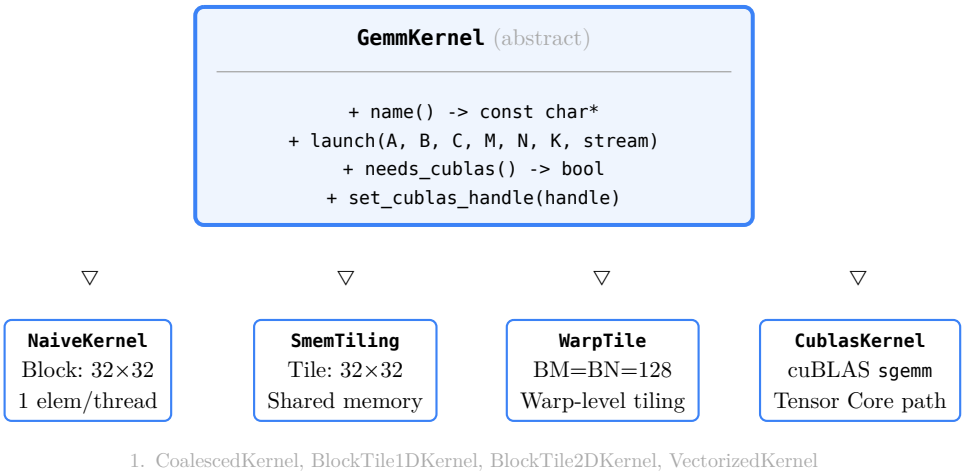
On H100 with NVLink, `AllReduce` latency is already very low (~ 1 ms for 256 MB), limiting overlap opportunity. This technique would likely be more beneficial on systems with higher communication latency (e.g., PCIe or cross-node InfiniBand).

5 Software Design

The codebase is organized into three loosely-coupled layers: a **kernel abstraction layer** with pluggable GEMM implementations, a **CUDA resource layer** with RAII wrappers, and the **tensor parallelism layer** that composes both.

5.1 Kernel Abstraction and Registry

All GEMM kernels—from the naive baseline to cuBLAS—implement a common abstract interface:



Listing 1: Kernel class hierarchy. Each concrete kernel encapsulates its CUDA `__global__` function, block/grid configuration, and launch logic. The abstract base provides a uniform `launch()` interface.

Each kernel file self-registers via a static initializer, eliminating the need for centralized dispatch tables:

```
// In naive.cu
class NaiveKernel : public GemmKernel { ... };
namespace { static int reg = KernelRegistry::add(
    std::make_unique<NaiveKernel>()); }
```

The `KernelRegistry` singleton manages all kernel instances. Adding a new kernel requires only creating a single `.cu` file—no header modifications or array synchronization needed. This *open-closed* design replaces the previous approach of parallel enum/function-pointer arrays that required manual synchronization across four separate data structures.

5.2 RAII Resource Management

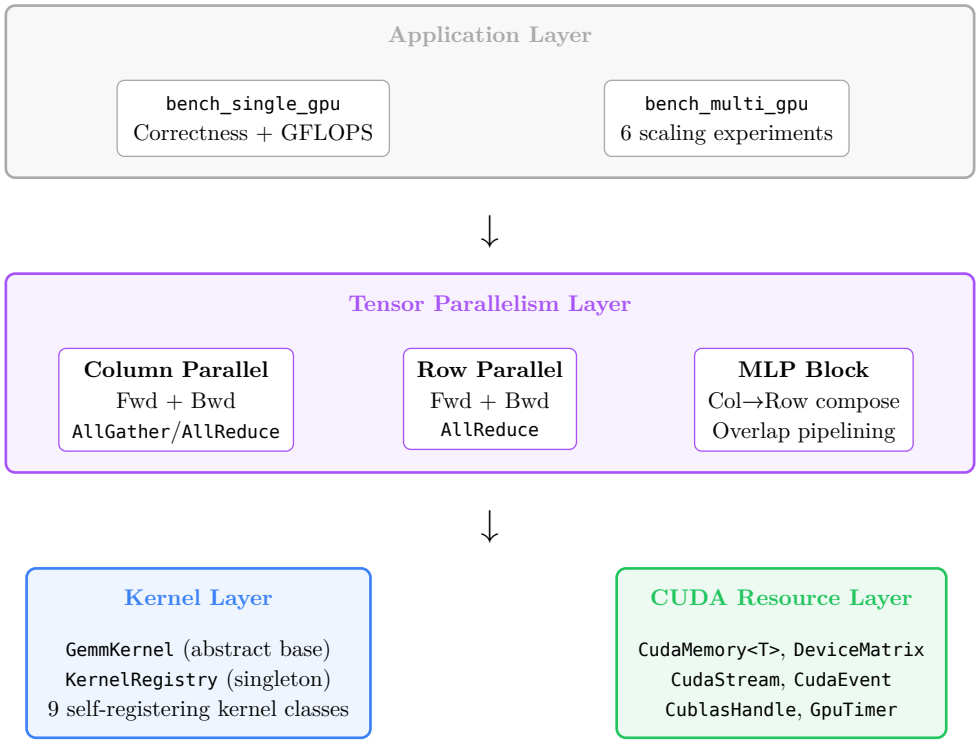
All GPU resources are managed via move-only RAII wrappers, ensuring deterministic cleanup and preventing leaks:

Class	Manages	Acquire	Release
`CudaMemory<T>`	Device allocation	`cudaMalloc`	`cudaFree`
`DeviceMatrix`	2D float matrix	Delegates to `CudaMemory`	Automatic
`CudaStream`	CUDA stream	`cudaStreamCreate`	`cudaStreamDestroy`
`CudaEvent`	CUDA event	`cudaEventCreate`	`cudaEventDestroy`
`CublasHandle`	cuBLAS context	`cublasCreate`	`cublasDestroy`

Table 5: RAII resource wrappers. All classes delete copy operations and implement move semantics, preventing accidental double-free or resource leaks.

`DeviceMatrix` composes `CudaMemory<float>` with row/column dimensions, providing `init_random()`, `zero()`, and host-transfer methods. The benchmark code uses `DeviceMatrix` throughout, reducing buffer management from ∼10 lines per matrix to a single constructor call.

5.3 Architecture Overview



Listing 2: System architecture. The three layers are loosely coupled: kernels are pluggable via the registry, RAII wrappers manage GPU resources, and the tensor parallelism layer composes both.

The tensor parallelism layer (Listing 2, middle) consumes kernels through the `const GemmKernel&` interface, remaining agnostic to the specific kernel implementation. Gradient GEMM operations (transpose + multiply) are factored into reusable `grad_gemm_at_b` and `grad_gemm_a_bt` helpers, reducing the ∼200 lines of duplicated backward-pass code to two composable primitives.

6 Conclusion

This project provides a comprehensive study of GEMM optimization and tensor parallelism across three dimensions:

Single-GPU kernel optimization. Seven progressively optimized CUDA kernels demonstrate the transition from memory-bound ($\sim 6\{,\}300$ GFLOPS) to compute-bound ($\sim 32\{,\}800$ GFLOPS) operation. The most impactful optimization is 2D register blocking, which reduces shared memory reads per FMA from

2 to 0.25. Our best custom kernel achieves 63% of cuBLAS throughput on FP32—the remaining gap is attributable to cuBLAS's Tensor Core utilization and auto-tuning.

Multi-GPU tensor parallelism. Scaling experiments on 8×H100 GPUs with matrices up to 32768×32768 demonstrate near-ideal strong scaling and quantify the communication-compute crossover: as local kernels approach peak efficiency, the communication-to-computation ratio rises from 0.20 (naive) to 0.85 (cuBLAS), directly measuring the diminishing returns of kernel optimization alone in distributed settings.

Software engineering. The codebase follows an object-oriented, RAII-based design with a self-registering kernel registry (open-closed principle), move-only resource wrappers (no manual `cudaFree`), and composable gradient GEMM helpers. This architecture enables adding new kernel implementations via a single file with zero modifications to existing code.

These results underscore that efficient distributed deep learning requires **co-optimization** of local compute kernels, system-level communication infrastructure, and clean software abstractions—a principle that motivates ongoing research in communication-avoiding algorithms, kernel fusion, and hardware interconnect design.

Bibliography

[1] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[2] Z. Jia, M. Maggioni, B. Staiger, and D. P. Scarpazza, "Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking," *arXiv preprint arXiv:1804.06826*, 2018.

[3] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[4] NVIDIA Corporation, "cuBLAS Library." 2026. [Online]. Available: https://docs.nvidia.com/cuda/pdf/CUBLAS_Library.pdf

[5] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.